Software Design Document

TIM Driver Module

Lab 02 – Microcontrollers and Embedded Systems

Module	TIM Driver
Version	1.0.0
Author(s)	J.M. Paez Sandoval, C.A. Martínez, D. Lopez Moreno, L.A. Lugo Muñiz
Institution	CETYS Universidad — Facultad de Ingeniería
Date	25/04/2026

1. Introduction

1.1 Purpose

This document describes the design and architecture of the tim_driver module. It provides details on data structures, enumerations, algorithms, and control flow to guide implementation and testing of the STM32F4 timer peripheral driver.

1.2 Scope

The tim_driver is a low-level bare-metal module for STM32F4xx timers. It provides APIs to initialize timers, start/stop counters, read and clear the Update Interrupt Flag (UIF), and configure PWM output modes via CCMRx registers. It is used by the timer module (TIM2 delays) and the pwm module (TIM1 PWM). Peripheral clock enabling is not performed by tim_driver — caller modules must enable the APBx clock before calling tim_init().

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
TIM	Timer peripheral (TIM1–TIM11 on STM32F411)
PSC	Prescaler register — divides timer clock: tick = CLK / (PSC+1)
ARR	Auto-Reload Register — counter wraps at ARR+1; shadow register
CNT	Counter register — current timer count value
UIF	Update Interrupt Flag (SR bit 0) — set on counter overflow
UEV	Update Event — triggered by EGR.UG; copies shadow registers to active
EGR.UG	Update Generation bit — forces immediate UEV (shadow copy)
CCMRx	Capture/Compare Mode Register — configures OC mode (PWM1/2)
OC1M	Output Compare Mode field — 110b=PWM Mode 1, 111b=PWM Mode 2
OC1PE	Output Compare Preload Enable — CCR1 updates on UEV only
ARPE	Auto-Reload Preload Enable (CR1 bit 7) — ARR double-buffered
CCER	Capture/Compare Enable Register — CCxE enables channel output
CCRx	Capture/Compare Register — duty cycle control value
BDTR.MOE	Main Output Enable (BDTR bit 15) — required for TIM1/TIM8 output
APB1	Advanced Peripheral Bus 1 — TIM2–TIM5 (16 MHz at reset)
APB2	Advanced Peripheral Bus 2 — TIM1, TIM9–TIM11 (16 MHz at reset)
TIM_Status_t	Return type: TIM_OK=0, TIM_ERROR=1

1.4 Requirements

All requirements are allocated in Docs/SRS/TIM_Driver_SRS.md (provided by the instructor without modification).

2. System Architecture

2.1 High-Level Design

The tim_driver sits at the lowest software layer, directly above the hardware registers. Peripheral clocks are not enabled here — the calling module is responsible.

Layer	Modules
Application	main.c — calls led, timer, pwm APIs; no register access
Middleware	timer.c (TIM2 delays), pwm.c (TIM1 PWM) — built on tim_driver
TIM Driver	All TIM register operations. Caller enables APBx clock first.
Hardware	STM32F411RE TIM1 (APB2) and TIM2 (APB1) via CMSIS TIM_TypeDef

2.2 Module Interfaces

Public API declared in tim_driver.h. Dependencies: stm32f411xe.h, stdint.h, stddef.h. Constants: TIM_APB1_CLOCK_HZ = 16000000U, TIM_APB2_CLOCK_HZ = 16000000U. No HAL.

3. Detailed Design

3.1 Class Diagram

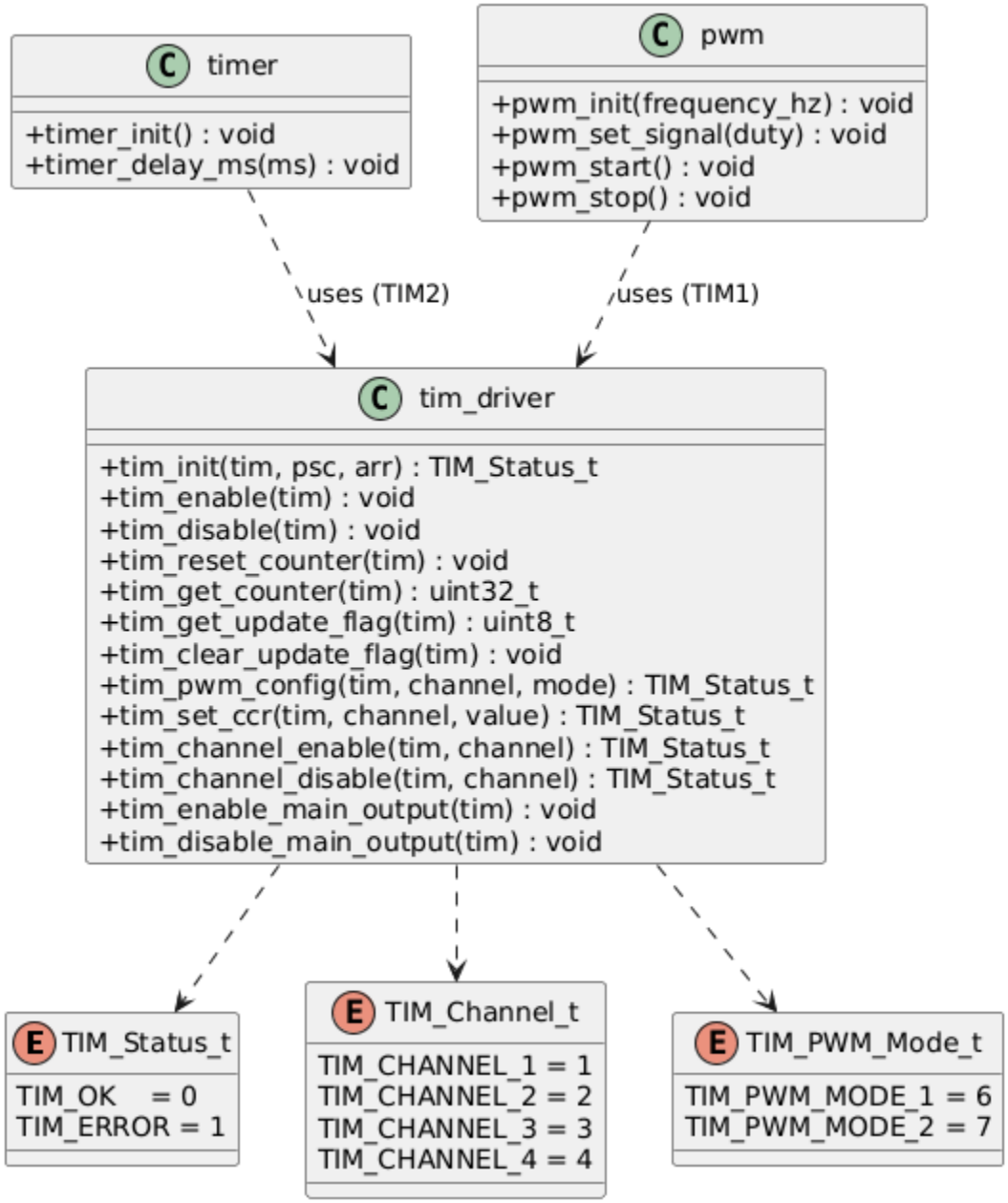

Figure 1. Class Diagram — tim_driver module, enumerations, and dependent modules (timer, pwm)

3.2 Module Behavior

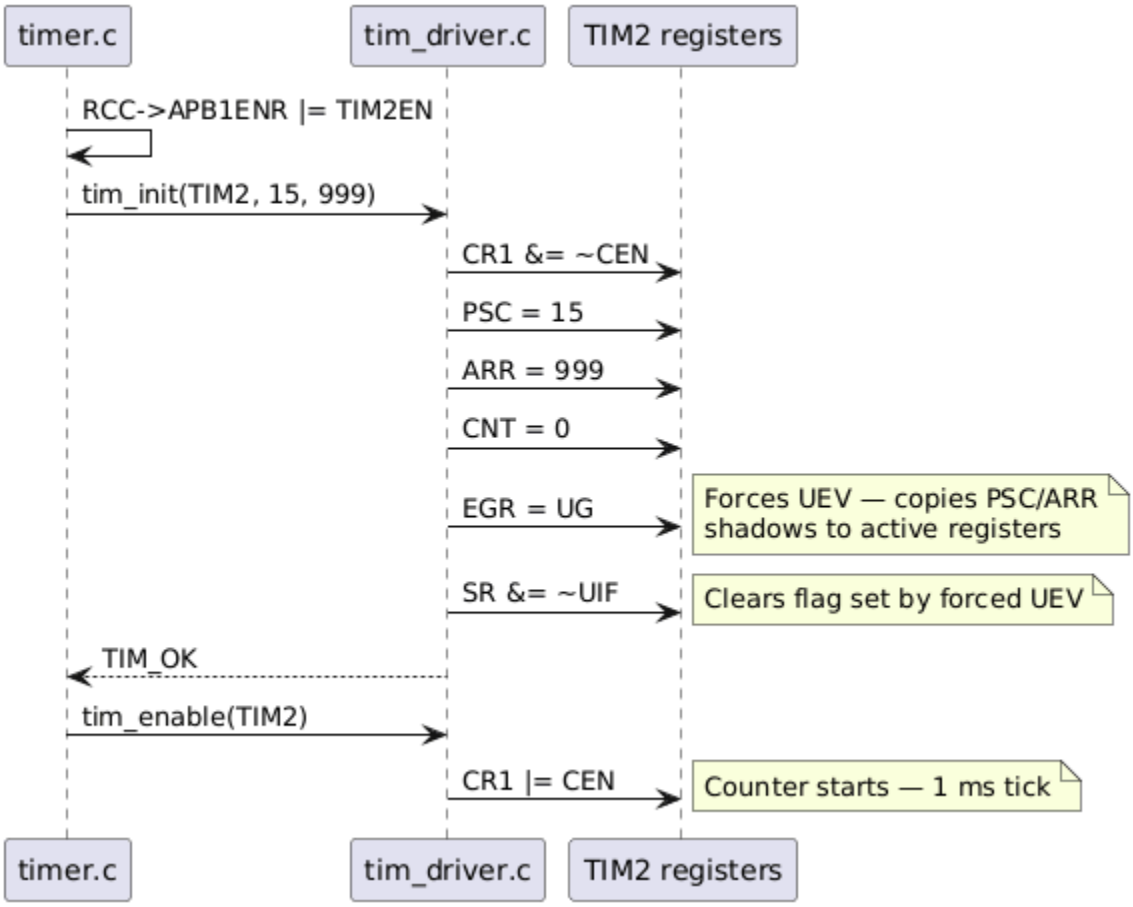

Figure 2. Sequence Diagram — tim_init() + tim_enable() for TIM2 1 ms tick

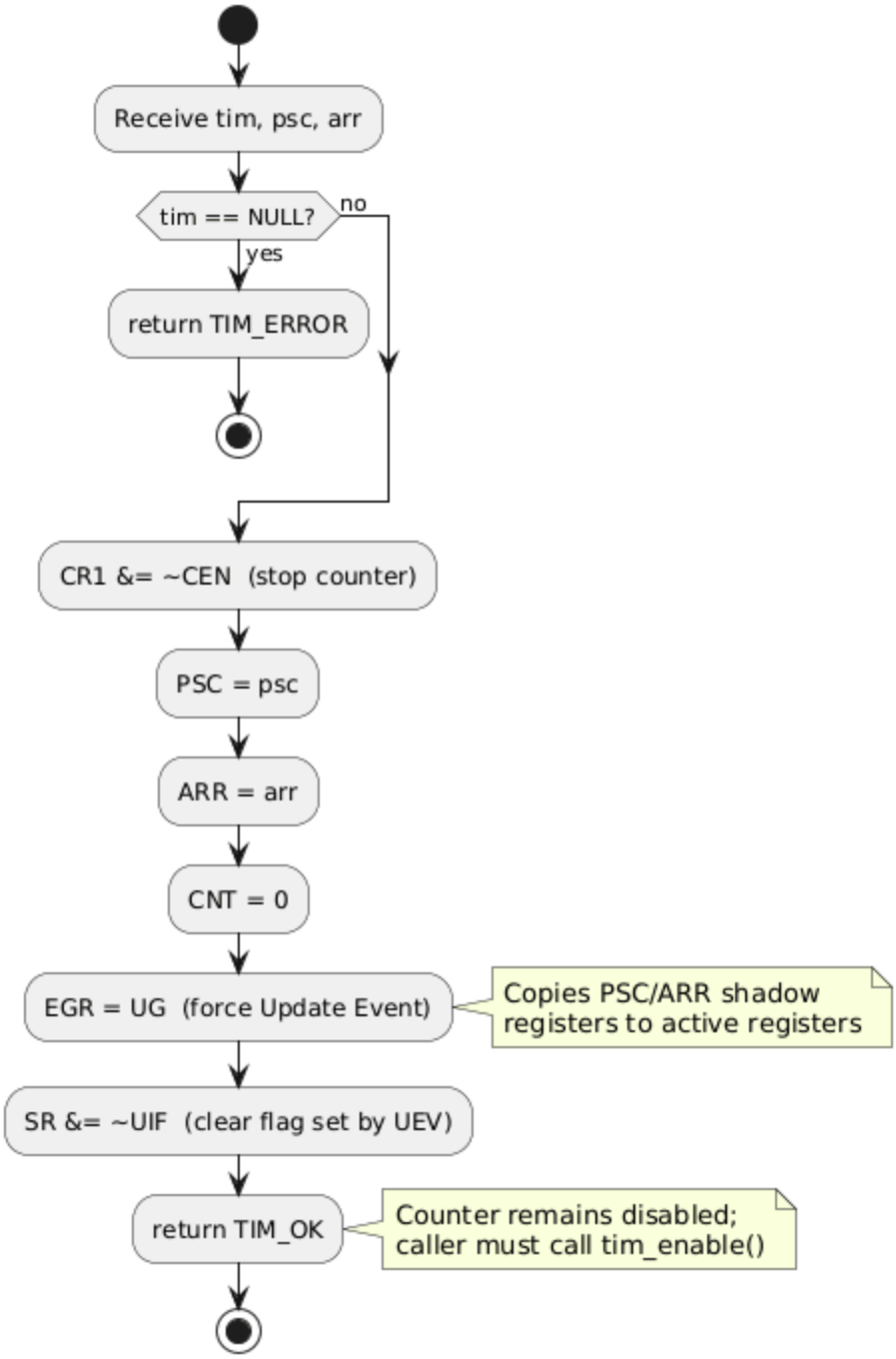

Figure 3. Activity Diagram - tim_init(): NULL check, PSC/ARR/CNT write, EGR.UG, UIF clear

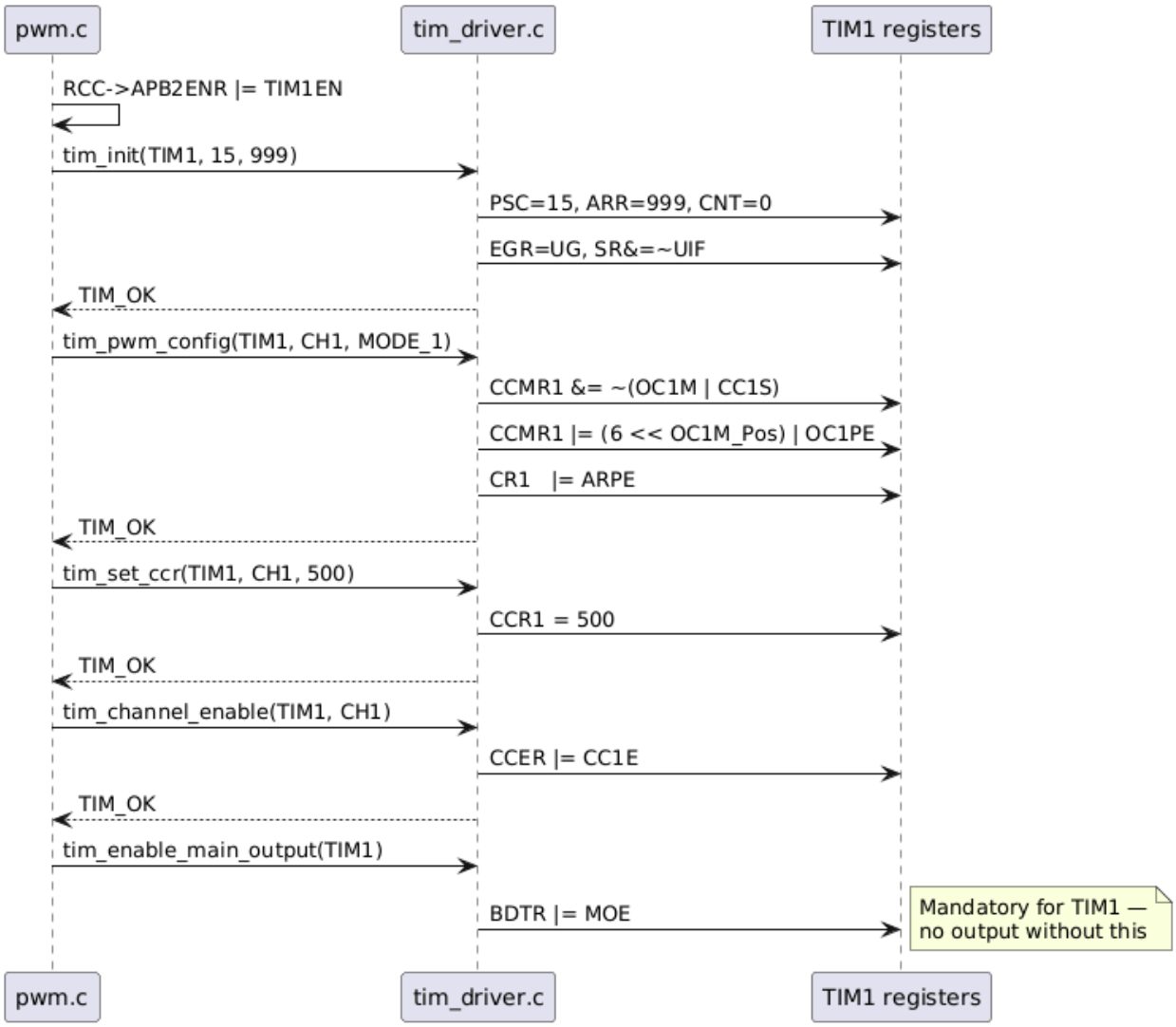

Figure 4. Sequence Diagram — tim_pwm_config() channel switch + tim_set_ccr() + tim_channel_enable() + MOE

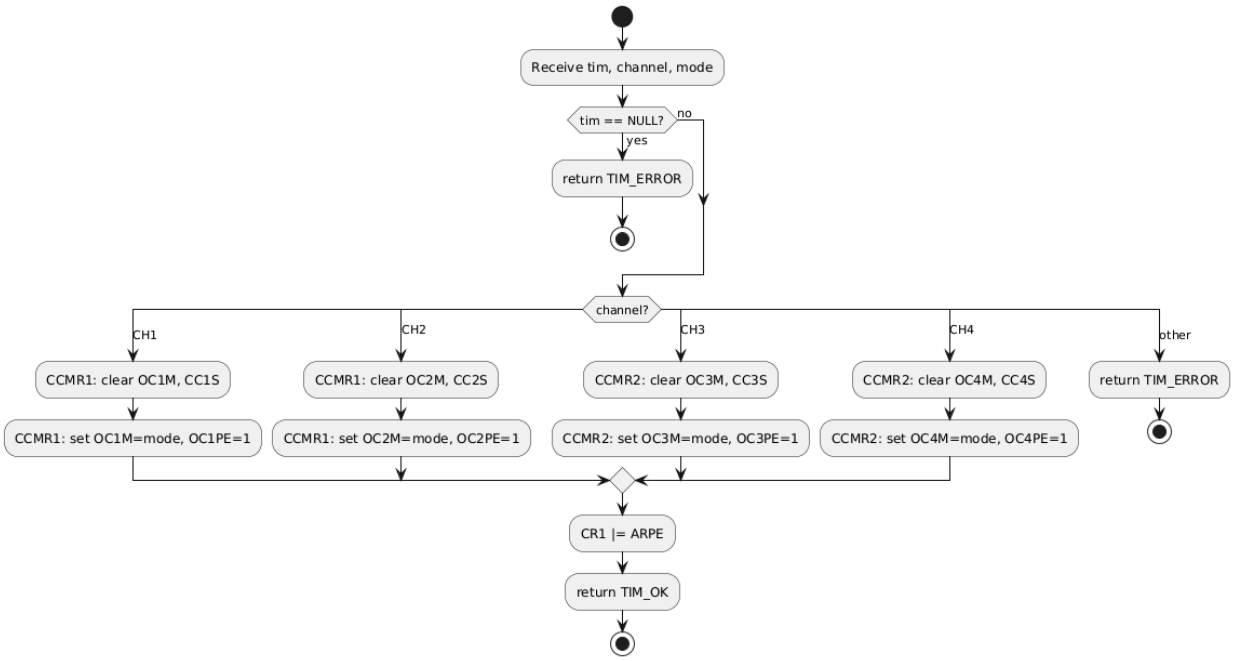

Figure 5. Activity Diagram - tim_pwm_config() channel selection switch

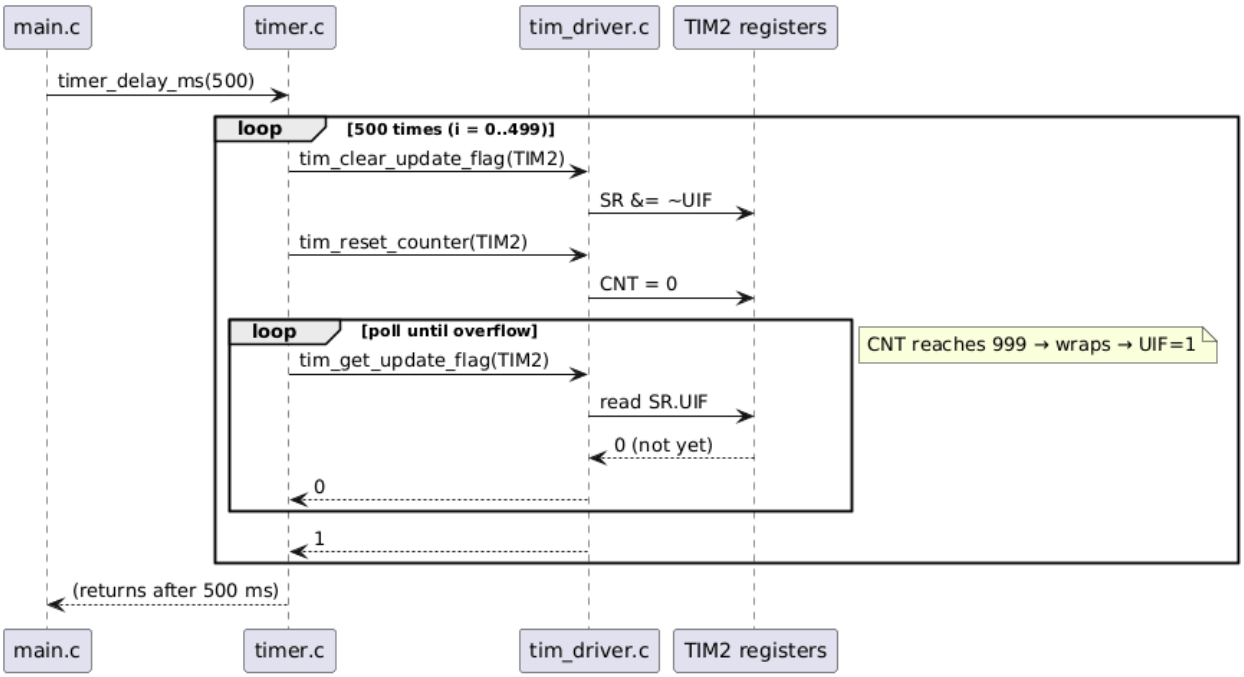

Figure 6. Sequence Diagram - timer_delay_ms() polling TIM2 UIF for 500 ms delay

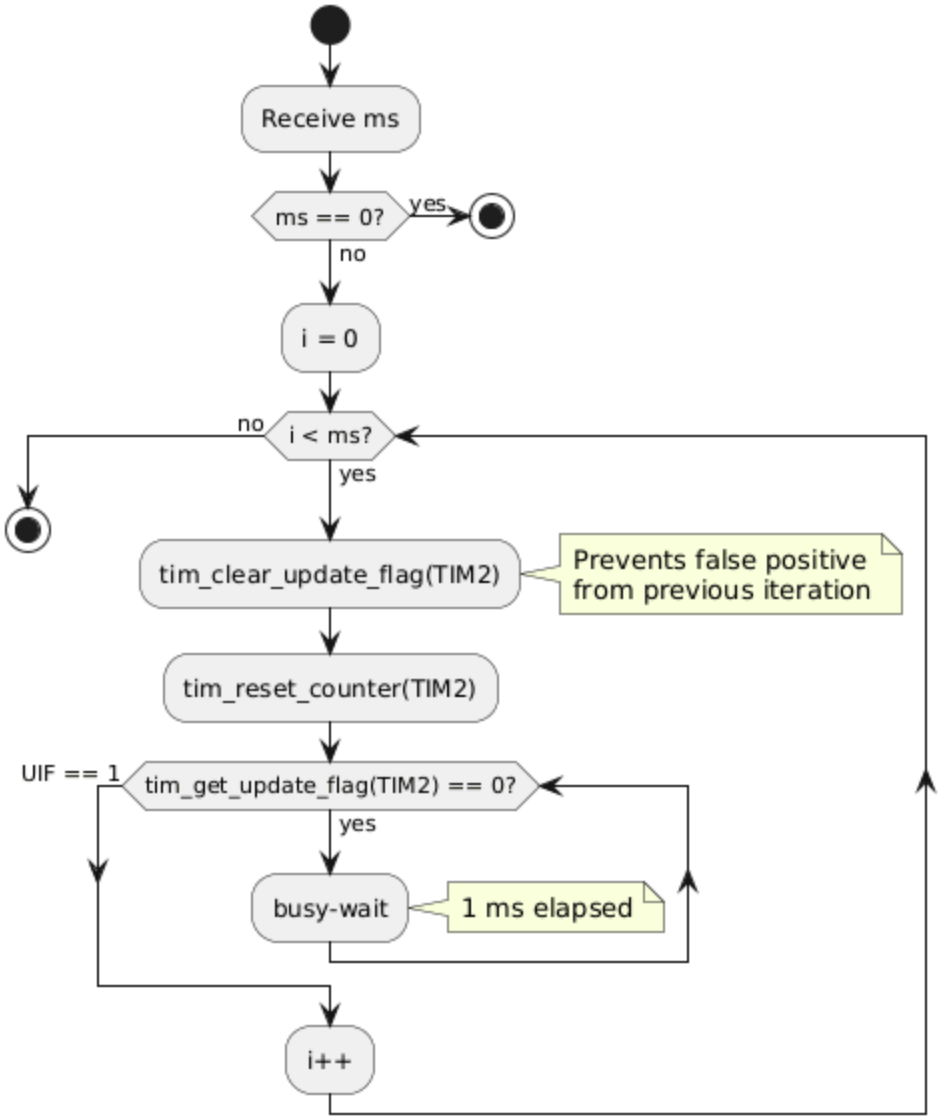

Figure 7. Activity Diagram - timer_delay_ms() busy-wait loop on SR.UIF

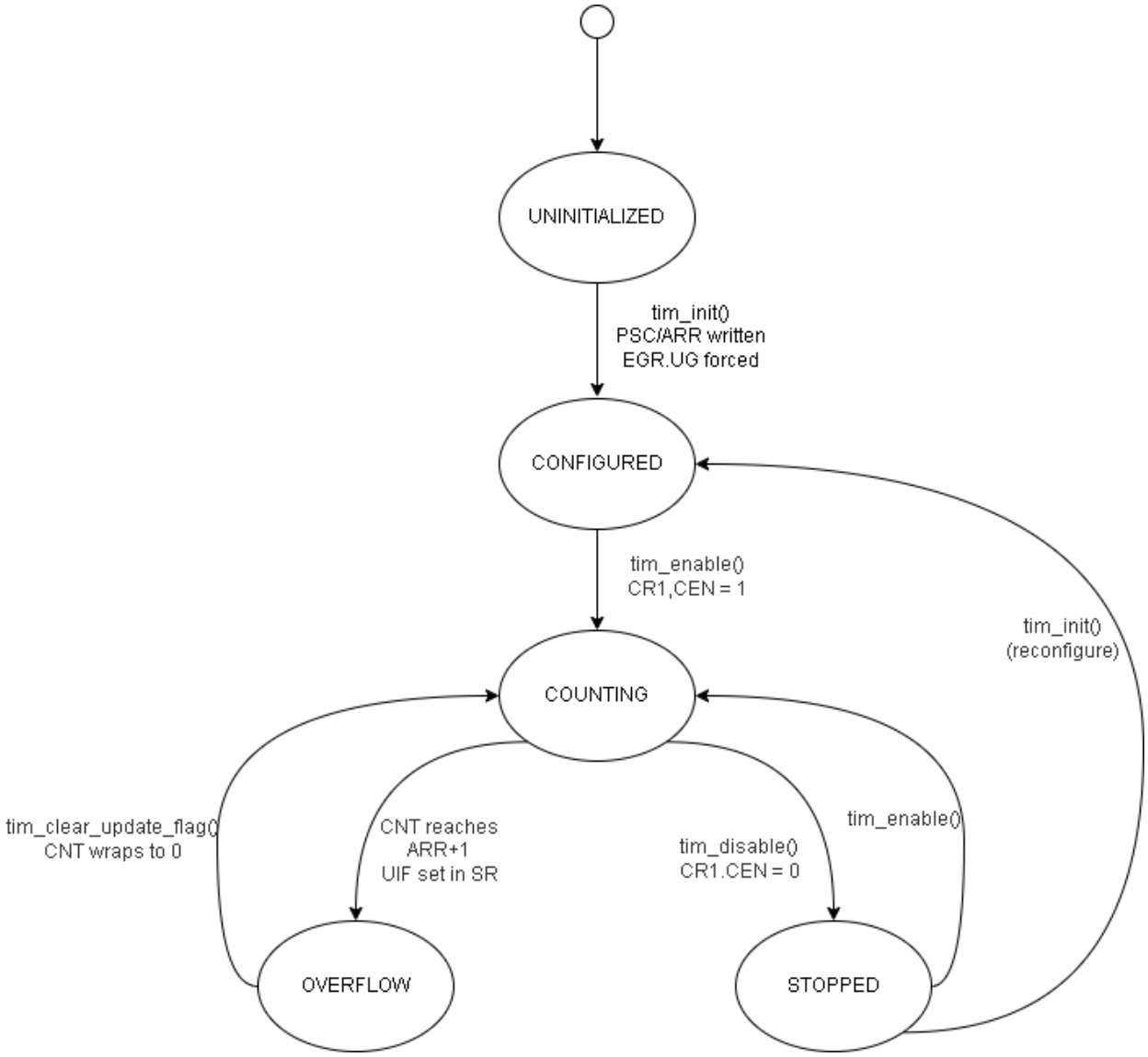

Figure 8. State Diagram - TIM states: UNINITIALIZED → CONFIGURED → COUNTING ↔ OVERFLOW / STOPPED

3.3 TIM Driver Module Description

tim_init() initializes a timer safely: stops the counter (CR1.CEN=0), writes PSC and ARR preloads, resets CNT, triggers a forced Update Event via EGR.UG (which copies shadow registers to active registers), then clears the UIF flag set by the forced UEV. The timer is left disabled; the caller must call tim_enable() to start counting.

tim_enable() / tim_disable() set or clear CR1.CEN. tim_reset_counter() writes zero to CNT. tim_get_counter() returns the raw CNT value.

tim_get_update_flag() reads SR.UIF: returns 1 if the counter has overflowed since the last clear, else 0. tim_clear_update_flag() writes 0 to SR.UIF (rc_w0 register). In timer_delay_ms(), UIF is cleared before resetting CNT to avoid a false-positive from a previous iteration.

tim_pwm_config() configures a channel in PWM output mode via a switch on the channel enum. For each channel it clears CCxS (output mode), sets the OCxM field to the requested PWM mode value (6=Mode 1, 7=Mode 2), enables preload (OCxPE=1), and sets CR1.ARPE.

tim_set_ccr() writes the CCRx register for the specified channel. With OC1PE set, the write goes to the preload register and becomes active at the next UEV — avoiding glitches during duty cycle changes.

tim_enable_main_output() sets BDTR.MOE (bit 15). This is mandatory for TIM1 and TIM8: without it, channel outputs remain disabled even with CCxE=1 and the counter running.

4. Application Programming Interfaces

4.1 Data Types and Macros

Enum	Values	Notes
TIM_Status_t	OK=0, ERROR=1	Return type; void functions have no return code
TIM_Channel_t	CH1=1, CH2=2, CH3=3, CH4=4	Channel index; maps to CCMRx/CCER bit position
TIM_PWM_Mode_t	MODE_1=6, MODE_2=7	Written to OCxM: 6=active-high, 7=active-low

Constant	Value	Notes
TIM_APB1_CLOCK_HZ	16000000U	TIM2–TIM5 input clock at HSI, APB1 pre=1
TIM_APB2_CLOCK_HZ	16000000U	TIM1, TIM9–TIM11 input clock at HSI, APB2 pre=1

4.2 Error Codes

Code	Value	Definition
TIM_OK	0	Operation completed successfully
TIM_ERROR	1	NULL timer pointer or invalid channel (outside 1–4)

4.3 Function Definitions

tim_init

Function Name	tim_init
Syntax	`TIM_Status_t tim_init(TIM_TypeDef *tim, uint16_t psc, uint32_t arr)`
Parameters (in)	tim — TIM peripheral pointer (TIM1, TIM2, …)psc — Prescaler value (0 = divide by 1)arr — Auto-reload value; counter wraps at arr+1

Parameters (inout)	None
Parameters (out)	None
Return value	TIM_OK or TIM_ERROR (if tim==NULL)
Description	Stops counter, writes PSC/ARR/CNT, forces UEV via EGR.UG to copy shadow registers, clears UIF. Timer is left disabled after this call.
Constraints	Caller must enable APBx clock before calling. Timer remains stopped; call tim_enable() to start.
Available via	tim_driver.h

tim_enable / tim_disable

Function Name	tim_enable / tim_disable
Syntax	`void tim_enable(TIM_TypeDef *tim)void tim_disable(TIM_TypeDef *tim)`
Parameters (in)	tim — TIM peripheral pointer
Parameters (inout)	None
Parameters (out)	None
Return value	void
Description	tim_enable: sets CR1.CEN=1 — counter starts. tim_disable: clears CR1.CEN=0 — counter freezes at current CNT value.
Constraints	No NULL check. tim_disable() does not reset CNT.
Available via	tim_driver.h

tim_reset_counter / tim_get_counter

Function Name	tim_reset_counter / tim_get_counter
Syntax	`void tim_reset_counter(TIM_TypeDef *tim)uint32_t tim_get_counter(TIM_TypeDef *tim)`
Parameters (in)	tim — TIM peripheral pointer
Parameters (inout)	None
Parameters (out)	None
Return value	void / uint32_t (current CNT value)
Description	tim_reset_counter: writes 0 to CNT; used in timer_delay_ms() to start a fresh 1 ms window. tim_get_counter: returns raw CNT register value.
Constraints	No NULL check. TIM2 has 32-bit CNT; other timers have 16-bit CNT.
Available via	tim_driver.h

tim_get_update_flag / tim_clear_update_flag

Function Name	tim_get_update_flag / tim_clear_update_flag
Syntax	`uint8_t tim_get_update_flag(TIM_TypeDef *tim)void tim_clear_update_flag(TIM_TypeDef *tim)`
Parameters (in)	tim — TIM peripheral pointer
Parameters (inout)	None
Parameters (out)	None
Return value	uint8_t (1=overflow occurred, 0=not yet) / void
Description	tim_get_update_flag: reads SR.UIF. tim_clear_update_flag: clears SR.UIF (rc_w0 — write 0 clears the bit).
Constraints	UIF is also set by the forced EGR.UG in tim_init() and cleared there automatically.
Available via	tim_driver.h

tim_pwm_config

Function Name	tim_pwm_config
Syntax	`TIM_Status_t tim_pwm_config(TIM_TypeDef *tim, TIM_Channel_t channel, TIM_PWM_Mode_t mode)`
Parameters (in)	tim — TIM pointerchannel — TIM_CHANNEL_1…4mode — TIM_PWM_MODE_1 or TIM_PWM_MODE_2
Parameters (inout)	None
Parameters (out)	None
Return value	TIM_OK or TIM_ERROR
Description	Clears CCxS (output mode), sets OCxM=mode, sets OCxPE=1 (preload), sets CR1.ARPE. Uses a switch on channel to select correct CCMR1/CCMR2 fields.
Constraints	Pin must be in AF mode before calling. Call tim_channel_enable() and tim_enable_main_output() (for TIM1) before starting the counter.
Available via	tim_driver.h

tim_set_ccr

Function Name	tim_set_ccr
Syntax	`TIM_Status_t tim_set_ccr(TIM_TypeDef *tim, TIM_Channel_t channel, uint32_t value)`

Parameters (in)	tim — TIM pointerchannel — TIM_CHANNEL_1…4value — 0 to ARR+1 for 0–100% duty
Parameters (inout)	None
Parameters (out)	None
Return value	TIM_OK or TIM_ERROR
Description	Writes value to CCRx for the specified channel. Duty (%) = value / (ARR+1) x 100. With OCxPE=1, activates on the next UEV.
Constraints	value > ARR+1 produces 100% duty. value = 0 produces 0% duty.
Available via	tim_driver.h

tim_channel_enable / tim_channel_disable

Function Name	tim_channel_enable / tim_channel_disable
Syntax	`TIM_Status_t tim_channel_enable(TIM_TypeDef *tim, TIM_Channel_t channel)TIM_Status_t tim_channel_disable(TIM_TypeDef *tim, TIM_Channel_t channel)`
Parameters (in)	tim, channel — TIM_CHANNEL_1…4
Parameters (inout)	None
Parameters (out)	None
Return value	TIM_OK or TIM_ERROR
Description	Sets or clears CCER.CCxE for the specified channel. Bit positions: CH1=bit0, CH2=bit4, CH3=bit8, CH4=bit12.
Constraints	For TIM1/TIM8, BDTR.MOE must also be set via tim_enable_main_output().
Available via	tim_driver.h

tim_enable_main_output / tim_disable_main_output

Function Name	tim_enable_main_output / tim_disable_main_output
Syntax	`void tim_enable_main_output(TIM_TypeDef *tim)void tim_disable_main_output(TIM_TypeDef *tim)`
Parameters (in)	tim — TIM pointer
Parameters (inout)	None
Parameters (out)	None
Return value	void

Description	Sets or clears BDTR.MOE (bit 15). Required for TIM1/TIM8: without MOE=1, channel pins are not driven regardless of CCxE or counter state.
Constraints	No NULL check. Must be called after tim_channel_enable() and before tim_enable() for TIM1.
Available via	tim_driver.h

4.4 Internal Function Definitions

The tim_driver module has no internal (static) functions. All logic is implemented directly within the public API functions.

5. Future Enhancements

- Add interrupt-driven UIF handling: expose tim_enable_update_irq() / tim_disable_update_irq() to set/clear DIER.UIE.
- Implement tim_set_frequency() and tim_set_duty_cycle() as high-level wrappers computing PSC/ARR/CCRx from Hz and percentage inputs.
- Add input capture mode configuration (CCxS = 01/10) for frequency measurement and pulse timing.
- Add one-pulse mode (OPM) support via CR1.OPM for single-shot timer events.